Day 9 – Custom Pipes & Component Interaction

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **IDE Ready**: Open VS Code, prepared to generate custom pipes and implement component communications.

2. **Prerequisites**:
   - You must have completed Day 8 successfully (understanding Angular Signals state management).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Pass data from a parent component to a child component using the `@Input` decorator.
- Bubble events from a child component up to a parent component using `@Output` and `EventEmitter`.
- Use template reference variables (`#var`) to reference HTML elements directly.
- Create a custom Pipe using the `@Pipe` decorator and `PipeTransform` interface.
- Register and use custom pipes inside standalone component templates.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define component hierarchy. Explain parent-child bindings and data transformation pipes.
2. Key Concepts & Core Ideas	7 min	Explain @Input properties, @Output events, transform pipelines, and pure pipes.
3. Live Coding Walkthrough	23 min	Create child components, wire data inputs and click events, generate custom format pipes, and apply to layouts.

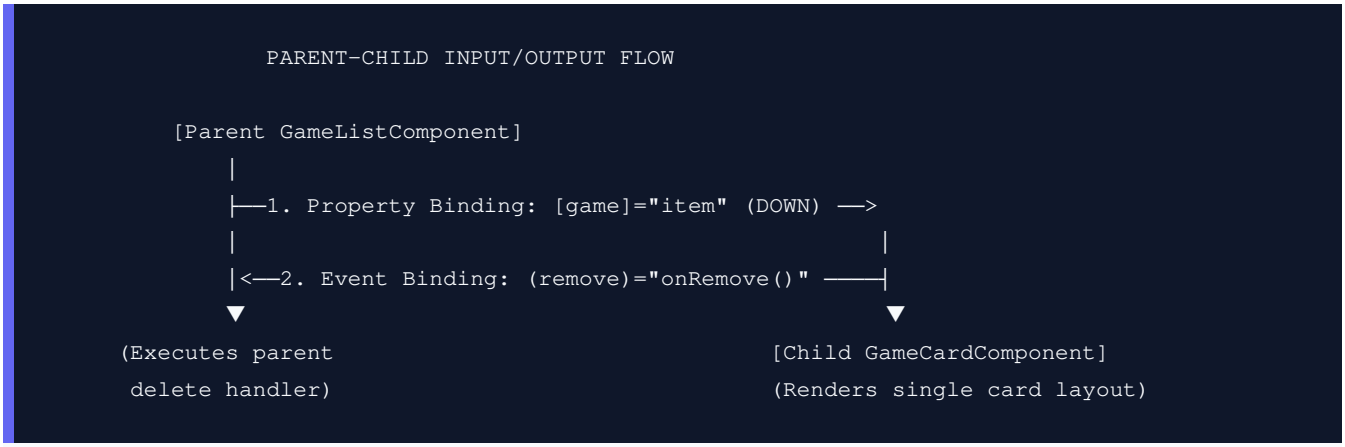
4. Check for Understanding	5 min	Q&A on property flow direction, EventEmitter types, and pure vs impure pipes.
5. Class Wrap-up & Checkpoint	3 min	Verify child interactions bubble up and custom pipes format views correctly.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

As components grow in size, they become difficult to maintain. To keep code clean, we break larger components down into smaller, focused child components. For example, rather than rendering the entire game details directly in the list, we can create a small `GameCardComponent` child. However, this child component needs a way to receive the game details from its parent, and notify the parent when a user clicks a button inside it.

- **Component Communication**: Parent components pass properties DOWN using **Property Binding** with `@Input`. Child components pass events UP using **Event Binding** with `@Output` and `EventEmitter`.
- **Data Formatting**: Sometimes, data in the database (like a raw key string `"abc123xyz456"`) needs to be formatted for display (like `"ABC1-23XY-Z456"`). Angular handles this using **Pipes**.



2. Key Concepts & Core Ideas (7 min)

Introduce these communication and piping architectures:

- **`@Input()`**: A decorator that defines a writable property on a child component. It allows a parent component to feed data down into the child.
- **`@Output()`**: A decorator that defines an event emitter property. It allows a child component to emit custom events up to the parent.
- **`EventEmitter`**: A utility class used by `@Output` properties to dispatch event payloads asynchronously.
- **Custom Pipe (`@Pipe`)**: A class decorated with `@Pipe` implementing the `PipeTransform` interface. It

transforms input parameters into a desired output format (e.g. converting a string to uppercase and adding custom hyphens).

- **Pure Pipe**: The default pipe type. It is only executed when the input parameters change, making it extremely performant.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Creating a Custom Pipe

- **Concept**: Generate a custom pipe called `key-format` that takes a raw game key string (e.g., `A3F9B21C4DE7`) and splits it into capitalized blocks separated by hyphens (e.g., `A3F9-B21C-4DE7`).
- **Code**: Generate pipe:

```
# Generate pipe under src/app/pipes/key-format.pipe.ts
ng generate pipe pipes/key-format
```

- **Verify**: Check that `src/app/pipes/` contains `key-format.pipe.ts` and `key-format.pipe.spec.ts` files.

Step 3.2: Implementing Pipe Transformation Logic

- **Concept**: Open `src/app/pipes/key-format.pipe.ts`. We will write the string manipulation logic inside the `transform()` method.
- **Code**: Modify `src/app/pipes/key-format.pipe.ts`:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'keyFormat',
  standalone: true // Standalone pipe can be imported directly
})
export class KeyFormatPipe implements PipeTransform {
  # The transform method takes the input value and optional parameters, returning the format
  transform(value: string | undefined): string {
    if (!value) return '';

    // Clean string: remove existing hyphens and convert to uppercase
    const cleanVal = value.replace(/-/g, '').toUpperCase();

    // Match groups of 4 characters using regex
    const matches = cleanVal.match(/.{1,4}/g);

    if (matches) {
      // Join the chunks back together with a hyphen
      return matches.join('-');
    }

    return cleanVal;
```

```
  }
}
```

Step 3.3: Creating a Child Component

- **Concept**: Generate a child component `game-card` that will render a single card layout, taking the game details as an input property and emitting a remove request on click.
- **Code**: Generate component:

```
ng generate component components/game-card
```

Modify `src/app/components/game-card/game-card.component.ts`:

```
import { Component, Input, Output, EventEmitter } from '@angular/core';
import { CommonModule } from '@angular/core';
import { Game } from '../../services/game.service';
import { KeyFormatPipe } from '../../pipes/key-format.pipe'; // Import custom pipe

@Component({
  selector: 'app-game-card',
  standalone: true,
  imports: [CommonModule, KeyFormatPipe],
  templateUrl: './game-card.component.html',
  styleUrls: ['./game-card.component.css']
})
export class GameCardComponent {
  // 1. @Input decorator allows parent component to pass down Game data
  @Input() game!: Game;

  // Mock key code to show pipe demonstration
  mockKey: string = 'a3f9b21c4de79901cc84';

  // 2. @Output decorator registers a custom event name
  // EventEmitter defines the data payload type (number) emitted
  @Output() remove = new EventEmitter<number>();

  // Emit event up to parent with game ID
  onRemoveClick(): void {
    this.remove.emit(this.game.id);
  }
}
```

Modify `src/app/components/game-card/game-card.component.html`:

```
<div class="card">
  <h3>{{ game.title }}</h3>
  <p>Price: ${{ game.price | number:'1.2-2' }}</p>

  <!-- Apply custom pipe to format key string -->
  <p>Demo Key: <code>{{ mockKey | keyFormat }}</code></p>
```

```
  <!-- Trigger output event on click -->
  <button (click)="onRemoveClick()" class="btn-remove">Remove from inventory</button>
</div>
```

Step 3.4: Incorporating Child Component and Custom Pipe in Parent

- **Concept**: Register the new `GameCardComponent` inside `GameListComponent` imports, and replace the inner list layout with our component selector.
- **Code**: Modify `src/app/components/game-list/game-list.component.ts` (import child):

```
import { GameCardComponent } from '../game-card/game-card.component'; // Import card component
...
@Component({
  ...
  imports: [CommonModule, FormsModule, RouterModule, GameCardComponent], // Register in import
  ...
```

Modify `src/app/components/game-list/game-list.component.html` (replace the inner `@for` loop structure to render child cards):

```
...
  @if (games.length === 0) {
    <p class="empty-msg">No games in inventory.</p>
  } @else {
    <div class="game-cards-container">
      @for (game of games; track game.id) {
        <!--
          [game]="game" passes the loop game down as an @Input.
          (remove)="onRemoveGame($event)" listens for the @Output event,
          passing the emitted number payload via $event.
        -->
        <app-game-card
          [game]="game"
          (remove)="onRemoveGame($event)">
        </app-game-card>
      }
    </div>
  }
...
```

Add the removal handler to `src/app/components/game-list/game-list.component.ts`:

```
...
export class GameListComponent implements OnInit {
  ...
  // Event handler matched to child output event trigger
  onRemoveGame(gameId: number): void {
    this.games = this.games.filter(g => g.id !== gameId);
```

```
      console.log(`[Parent Component] Removed game ID: ${gameId}`);
    }
  }
```

- **Verify**: Open `http://localhost:4200/`. Confirm that the catalog list displays individual card components. Check that the "Demo Key" is formatted cleanly with dashes as `A3F9-B21C-4DE7-9901-CC84`. Click "Remove from inventory" on a card and check that the card disappears from the parent list.

---

4. Check for Understanding (5 min)

> ***Question***: *"What does the `$event` keyword represent inside parent templates when binding to custom child events? e.g. `(remove)="handler($event)"`?"*
> ***Answer***: *`$event` is a special variable that represents the data payload emitted by the child's `EventEmitter`. In our code, since `remove` was declared as `EventEmitter<number>`, `$event` passes the emitted `gameId` number value down to the parent's handler method.*

> ***Question***: *"Why does Angular recommend using custom Pipes for formatting view data instead of calling component class helper methods directly inside templates? e.g. `{{ formatKey(key) }}`?"*
> ***Answer***: *Calling class methods directly inside template interpolation is a performance bottleneck. Angular cannot track if method return values change, so it executes the method on every single change detection cycle (thousands of times). Pipes (which are pure by default) cache their results and are only executed when their input values change.*

> ***Question***: *"Can you pass parameters to a custom pipe inside templates? If so, what is the syntax?"*
> ***Answer***: *Yes. You pass parameters to a pipe by appending a colon `:` followed by the parameter value. For example, in `{{ value | myPipe:param1:param2 }}`, `param1` and `param2` are sent as arguments to the pipe's `transform()` method after the main value.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ The `GameCardComponent` is successfully created and handles `@Input` properties.
  - ○ The child component emits a `remove` EventEmitter output which is captured by the parent component.
  - ○ The custom `keyFormat` pipe correctly parses raw strings and injects hyphens.